CAHIER D'ARCHITECTURE · VERSION 0.1 · 28 SEPTEMBRE 2026

Un système universel inspiré de l'eau

Noyau en Rust · démarrage depuis une clé USB

développeur interne pour les pilotes · autonomie encadrée

Projet de Jhonny

Document de travail : architecture, différenciation et feuille de route.

La nouveauté annoncée est à démontrer ; voir la section 14 (recherche d'antériorité du 28/09/2026).

Sommaire

Résumé . . . 3
1. Vision et principes . . . 3
2. Réalité du matériel : natif ou hôte . . . 4
3. Vue d'ensemble . . . 5
4. La Molécule (noyau) . . . 5
5. Les Courants (communication) . . . 6
6. Les Contenants (matériel et pilotes) . . . 6
7. La Forge (développeur interne) . . . 7
8. La Charte (autonomie encadrée) . . . 8
9. Le Bassin (stockage et état) . . . 9
10. Démarrage depuis une clé USB . . . 9
11. Performance et fluidité . . . 10
12. Sécurité et modèle de menaces . . . 10
13. Applications, runtime et mode hôte . . . 10
14. Différenciation par rapport aux OS existants . . . 11
15. Feuille de route . . . 12
16. Risques et décisions ouvertes . . . 13
Annexe A. Glossaire . . . 13
Annexe B. Références . . . 13

Résumé

Ce document décrit l'architecture d'un système d'exploitation universel écrit en Rust. Il est pensé comme l'eau : une molécule qui ne change pas (le noyau) et une forme qui épouse chaque contenant (chaque appareil). Il démarre depuis une simple clé USB sur les PC x86-64, ARM64 et RISC-V, et vit comme une application (mode hôte) sur les appareils dont le matériel est fermé.

- **Molécule** : un micro-noyau minimal et immuable, d'environ vingt appels système.
- **Courants** : tout le reste tourne en services isolés qui communiquent par des canaux typés et observables.
- **Forge** : un développeur interne qui écrit, teste et corrige les pilotes en Rust, sous des portes de vérification qu'il ne peut pas modifier lui-même.
- **Charte** : une politique signée qui encadre son autonomie. Le système choisit *comment* ; l'humain fixe *quoi* et *jusqu'où*.
- **Clé amphibie** : un exécutable UEFI par architecture, système chargé en RAM, disques du PC hôte intouchés par défaut.

Ce qui est réellement différent. Les briques de base existent déjà chez Fuchsia, HarmonyOS, Redox, Asterinas et Theseus (micro-noyau, capacités, Rust, évolution à chaud). Ce qui n'existe pas, à ma connaissance : une Forge intégrée à l'OS et régie par une Charte appliquée par le noyau, sur une clé USB multi-architecture qui apprend le matériel qu'elle rencontre. C'est là qu'il faut concentrer l'originalité (section 14).

Réalité du chantier. Un OS natif pour tous les appareils est un projet de plusieurs années. Ce cahier propose un ordre réaliste pour une personne seule : QEMU d'abord, quelques pilotes standards, puis la Forge sur des périphériques virtuels, et seulement ensuite le matériel réel (section 15).

1. Vision et principes

L'eau garde la même molécule et change de forme selon le contenant. Chaque propriété de l'eau se traduit ici en règle de conception.

Propriété de l'eau	Règle de conception	Composant
Molécule fixe, forme variable	Noyau invariant, périphérie adaptable	Molécule, Contenants
Suit la pente, contourne les obstacles	Flux dynamiques, chemins de secours	Courants
Solvant universel	Interfaces typées et ouvertes	Courants, SDK de pilotes
Trois états	Données au repos, en mouvement, en mémoire	Bassin, Courants, Vapeur (cache)
Vases communicants	Équilibrage automatique de la charge	Ordonnanceur, Courants
Débordement	Quotas et contre-pression	Écluses
Cycle de l'eau	Boucle de rétroaction : mesures, correction	Forge, Charte

Objectifs

- **Universel** : natif sur x86-64, ARM64 et RISC-V ; mode hôte sur les appareils fermés.
- **Autonome** : diagnostic et réparation locale, sans réseau, dans le périmètre autorisé.
- **Simple à manœuvrer** : une configuration, une image, un retour arrière.
- **Rapide et fluide** : interface visible en moins de 5 s après le firmware (objectif à mesurer).
- **Sûr** : mémoire sûre, pilotes isolés, autonomie bornée par la Charte.
- **Différent** : par la Forge, la Charte et la clé amphibie (section 14).

Hors périmètre initial

- Compatibilité POSIX, Linux ou Windows dans le noyau (possible plus tard, en service séparé).
- Remplacer iOS ou HarmonyOS sur leur matériel verrouillé.
- Écrire seul les pilotes GPU et Wi-Fi propriétaires dès la première phase.

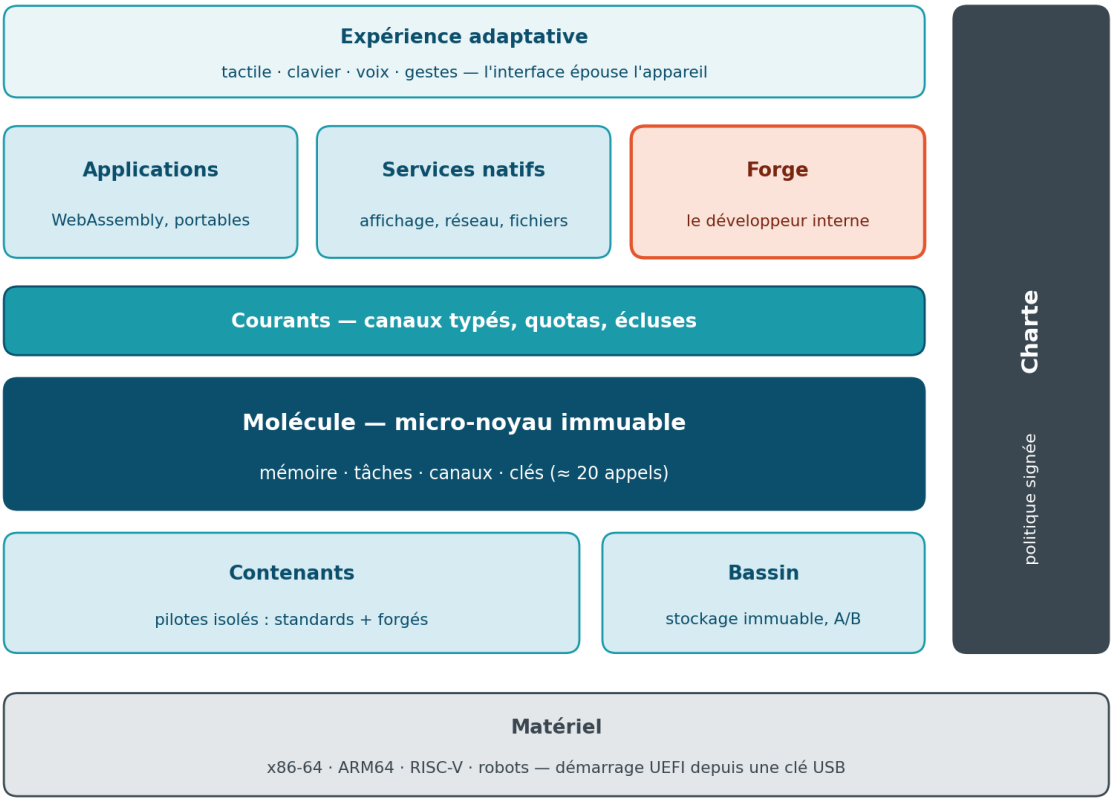
2. Réalité du matériel : natif ou hôte

Tous les appareils n'acceptent pas un système tiers. Le système existe donc sous deux formes : **natif** (il est l'OS) sur le matériel ouvert, et **hôte** (il tourne comme application avec son propre runtime) sur le matériel fermé. Ce sont les deux façons dont l'eau épouse un contenant : elle le remplit, ou elle vit à l'intérieur.

Appareil	Démarrage tiers	Mode visé	Remarque
PC x86-64 (UEFI)	Oui	Natif	Secure Boot : signature ou désactivation
PC et cartes ARM64	Oui, selon le firmware	Natif	Cartes sans UEFI : firmware UEFI ajouté ou chargeur dédié
RISC-V	Oui, selon la carte	Natif	Écosystème jeune, matériel limité
Chromebook	Souvent, en mode développeur	Natif ou hôte	Dépend du modèle et du firmware
Android (Pixel, autres)	Selon le fabricant	Hôte d'abord	Pilotes propriétaires, bootloader souvent verrouillé
Huawei	Souvent verrouillé	Hôte	Écosystème HarmonyOS fermé
iPhone, iPad	Non	Hôte uniquement	Règles de l'App Store
Mac Apple Silicon	Autorisé, pilotes non documentés	Hôte d'abord	Le natif exige de la rétro-ingénierie
Humanoïdes	Selon le constructeur	Natif si ouvert, sinon hôte	Actionneurs : classe R3 (jamais autonome)

Statut général à vérifier modèle par modèle. Règle de conception : le même code de services et d'applications doit tourner dans les deux modes ; seule la couche Contenants (ou son pont vers l'hôte) change.

3. Vue d'ensemble



Composant	Rôle	Confiance
Molécule	Micro-noyau : mémoire, tâches, canaux, clés	Base de confiance (signée, immuable)
Charte	Politique d'autonomie signée, appliquée par le noyau	Base de confiance
Courants	Communication typée, quotas, écoute	Fournis par la Molécule
Contenants	Pilotes isolés en espace utilisateur	Non fiables, confinés
Bassin	Stockage immuable, A/B, instantanés, persistance	Service vérifié
Forge	Génère, teste, promeut, surveille les pilotes	Non fiable, bornée par les portes
Services natifs, applications	Affichage, réseau, fichiers, applications WebAssembly	Isolés par clés
Expérience adaptative	Interface qui s'adapte à l'appareil et au mode d'entrée	Isolée

4. La Molécule (noyau)

La Molécule ne fournit que cinq abstractions : objets mémoire, tâches, canaux, clés (capacités), événements et budgets. Tout ce qui peut vivre en dehors du noyau y vit.

Famille	Appels (indicatifs)	Rôle
Mémoire (4)	mem_creer, mem_mapper, mem_liberer, mem_partager	Objets mémoire, partage sans copie
Tâches (3)	tache_creer, tache_arreter, tache_attendre	Isolation et supervision
Canaux (4)	canal_ouvrir, canal_envoyer, canal_recevoir, canal_fermer	Courants (section 5)
Clés (4)	cle_deriver, cle_restreindre, cle_revoquer, cle_transferer	Capacités : droits explicites et révocables

Famille	Appels (indicatifs)	Rôle
Événements (3)	irq_lier, minuterie_armer, attendre_evenements	Interruptions et temps
Budgets (2)	budget_creer, budget_attribuer	Temps CPU et mémoire bornés

Règles de construction

- Rust `no_std`. Le `unsafe` est confiné à un module `fondation` (registres, tables de pages, assembleur) ; le reste du noyau est en `#![forbid(unsafe_code)]`. Asterinas suit un principe voisin (« framekernel ») et annonce environ 14 % du code dans sa base de confiance mémoire.
- Multi-architecture dès le premier jour via un trait `Arch` (x86_64, aarch64, riscv64) ; jamais de code générique qui suppose x86.
- Aucun pilote dans le noyau, hormis le strict nécessaire : contrôleur d'interruptions (APIC, GIC, PLIC) et minuterie.
- Objectif de taille : moins de 15 000 lignes. Pour comparaison, Zircon (Fuchsia) prévoyait environ 100 appels système et Asterinas en implémente plus de 210 pour l'ABI Linux.
- Vérification : tests, model checking ciblé (par exemple Kani) et preuves sur l'allocateur de pages et le contrôle des clés (par exemple Verus).
- La Molécule est signée. Aucun composant, y compris la Forge, ne peut la remplacer sans signature humaine.

5. Les Courants (communication)

Les Courants sont le seul moyen de communication entre tâches. Un Courant est un canal **typé**, avec quota, que l'on peut **écouter**.

- **Typés** : le schéma versionné est vérifié à l'ouverture ; un schéma incompatible est refusé. Fuchsia fait de même avec ses protocoles typés (FIDL) : c'est un emprunt assumé.
- **Écluses** : chaque Courant a une capacité. Quand il est plein, l'émetteur reçoit de la contre-pression, pas une perte silencieuse ni une mémoire infinie.
- **Zéro copie** : les gros blocs passent par mémoire partagée (anneaux) ; seuls les descripteurs circulent.
- **Observables par construction** : un service autorisé peut écouter un Courant en lecture seule. Cela fournit journal, débogage et données pour la Forge sans modifier les services.
- **Pas d'espace de noms global** : un service ne voit que les Courants dont il détient la clé. Pas de racine « / » commune, pas d'utilisateur ambiant.

```
// pseudo-code : la définition d'un Courant fait foi, le noyau ne voit que des octets et un schéma
#[courant(version = 1, quota = 64)]
pub enum Clavier {
    Touche  { code: u16, appuyee: bool },
    Verrous { majuscules: bool, num: bool },
}
```

6. Les Contenants (matériel et pilotes)

Le matériel est décrit comme un **graphe** (Profil matériel), construit à partir de PCI, ACPI ou du device tree. Chaque périphérique est confié à un **Contenant** : un pilote qui tourne en espace utilisateur, avec des clés à portée limitée (plages MMIO, interruptions, fenêtres DMA derrière l'IOMMU).

Pilotes standards livrés

Quelques standards couvrent la majorité des PC : framebuffer UEFI (GOP), ACPI, PCIe avec NVMe, SATA/AHCI, USB xHCI avec HID et stockage de masse, virtio (machines virtuelles), Ethernet courant, horloge. Le Wi-Fi et le GPU accéléré dépendent des fabricants : prévoir des solutions de repli (Ethernet, partage de connexion USB, framebuffer).

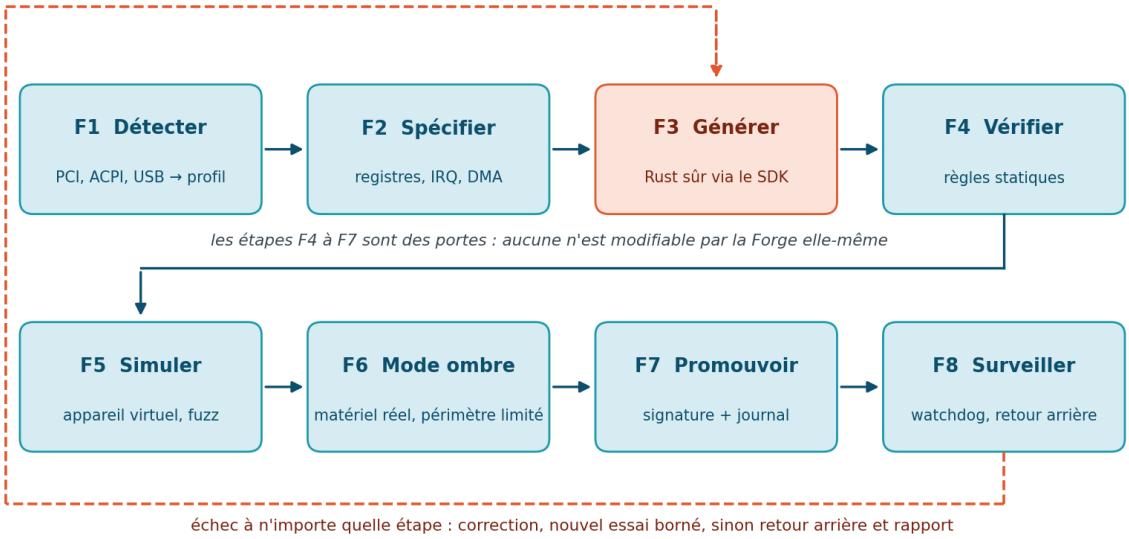
```
// pseudo-code : le contrat d'un Contenant (SDK de pilotes)
pub trait Contenant {
    const RISQUE: Risque;                              // R0..R3, vérifié au chargement
    fn sonder(&mut self, p: &ProfilMateriel) -> Result<Lien, Erreur>;
    fn courant_entree(&mut self, c: Courant<'_>) -> Result<(), Erreur>;
    fn irq(&mut self, cause: Irq) -> Result<(), Erreur>;
    fn arreter(&mut self);                             // toujours appelé avant retrait
}
// l'accès aux registres passe par une clé bornée au contrat : hors plage = erreur, pas de plantage
let regs = lien.mmio(Plage::depuis_contrat("nvme.bar0")?)?;
```

Classes de risque

Classe	Exemples	Règle pour la Forge
R0 diagnostic	Lecture d'informations, capteurs	Autonome dès le niveau A2
R1 non critique	Clavier, souris, audio, webcam	Autonome dès A3, après les portes
R2 données ou disponibilité	Stockage, réseau, GPU	A4 seulement, validation humaine initiale
R3 critique ou irréversible	Firmware, alimentation, horloges, contrôleurs flash, actionneurs de robot	Jamais autonome : humain et signature

7. La Forge (développeur interne)

La Forge transforme « matériel inconnu » ou « pilote défaillant » en « pilote Rust validé », sans humain sur place, mais jamais sans portes. Elle fait aussi les corrections internes : service qui plante, pilote qui dérive.



Étape	Ce qui se passe	Critère de sortie
F1 Détecter	Lit PCI, ACPI, USB ; cherche un pilote connu dans le magasin local signé	Pilote connu chargé, sinon F2
F2 Spécifier	Assemble un contrat : registres, interruptions, DMA, documents locaux, comportement observé passivement	Contrat complet ; sinon arrêt (pas de devinette sur R2 et R3)
F3 Générer	Le modèle écrit un Contenant en Rust sûr via le SDK ; `unsafe` interdit hors du wrapper MMIO audité	Le code compile
F4 Vérifier (porte)	Analyse statique : pas d'unsafe, plages de registres dans le contrat, boucles bornées, pas d'allocation en IRQ	Zéro violation
F5 Simuler (porte)	Appareil virtuel (QEMU, virtio ou modèle généré), tests de propriétés, fuzzing	Zéro échec sur N cas, couverture minimale

Étape	Ce qui se passe	Critère de sortie
F6 Mode ombre (porte)	Sur matériel réel, périmètre limité, IOMMU actif, interruptions observées	Aucun écart pendant une durée fixée
F7 Promouvoir (porte)	Signature par la clé de l'appareil ; journal (hash du code, modèle, résultats)	Politique de la Charte satisfaite
F8 Surveiller	Watchdog et compteurs ; en cas de défaut, retour à la version précédente et retour en F3 avec le rapport	Stabilité sur la durée

Fiabilité : ce que disent les travaux récents

La génération de pilotes par modèle de langage progresse, mais reste loin d'être fiable. Sur le banc DriveBench, AutoDriver (adaptation de pilotes Linux aux évolutions du noyau) n'obtient une compilation réussie que dans 56,4 % des 55 cas évalués. DevGen génère des modèles d'appareils virtuels utilisables pour 44 pilotes PCI/PCIe sur 50, ce qui montre l'intérêt de tester sans matériel. Conclusion de conception : supposer qu'un essai sur deux échoue. La vraie garantie, ce sont les portes F4 à F7, pas le modèle.

Autres règles

- **La Forge est un accélérateur, jamais une dépendance de démarrage.** Elle accepte un modèle local, un modèle distant ou aucun modèle ; sans elle, le système fonctionne avec les pilotes standards.
- **Trois niveaux de réparation** : redémarrage supervisé d'un service ; retour arrière A/B ; correctif généré par la Forge (mêmes portes). La Molécule et la Charte ne se réparent que par mise à jour humaine signée.
- **Ligne rouge** : la Forge ne modifie ni ses propres portes, ni la Charte, ni la Molécule. Une analyse de sécurité des systèmes auto-évolutifs (arXiv:2606.23075) montre que lorsque le même système propose et évalue ses changements, les défenses s'effondrent et les attaques deviennent persistantes.
- **Entrées non fiables** : toute chaîne venant du matériel, du firmware ou d'un document est traitée comme une donnée, jamais comme une instruction pour le modèle (injection).

8. La Charte (autonomie encadrée)

Principe : **le système choisit comment ; l'humain fixe quoi et jusqu'où.** « Décider ce qu'il veut » se traduit par des Intentions déclarées (« le Wi-Fi doit fonctionner », « démarrer vite ») et une vraie latitude sur les moyens, à l'intérieur d'un périmètre signé. Cette liberté est réelle, et c'est justement ce qui rend le périmètre acceptable pour quelqu'un qui branche la clé sur sa machine.

Niveau	Le système peut	Confirmation humaine
A0 Observer	Profil matériel et journal ; aucun code généré n'est chargé	Sans objet
A1 Proposer (défaut)	Générer et tester en simulation, présenter le résultat	Pour tout chargement
A2 Essayer	Mode ombre sur R0 et R1	Pour la promotion
A3 Appliquer	Promouvoir R0 et R1 après les portes	R2 : oui
A4 Étendu	Promouvoir R0 à R2 après les portes	R3 : toujours

Règles non négociables

- La Molécule, la Charte et les portes ne changent que par signature humaine.
- Disques internes du PC hôte : lecture seule par défaut ; écriture avec consentement explicite, par disque et par session.
- Retirer la clé laisse la machine hôte dans son état initial : rien n'est écrit dans le firmware sans demande explicite.
- R3 n'est jamais autonome ; les actionneurs de robot en font partie.
- Le journal est en ajout seul et signé ; le système ne peut pas l'effacer.
- Un interrupteur d'arrêt (combinaison de touches ou bouton physique) ramène immédiatement en A0.

9. Le Bassin (stockage et état)

- Racine système en lecture seule, adressée par contenu (hash), organisée par architecture.
- Mises à jour atomiques A/B : on prépare l'image B, on bascule, et après N échecs de démarrage on revient à A.
- Instantanés de la configuration et des pilotes forgés ; retour arrière en une opération.
- Partition DATA sur la clé : profils matériels, pilotes forgés signés, journal, configuration ; chiffrement optionnel.
- **Vapeur** : cache en RAM des profils et des pilotes chargés, recalculable à tout moment.
- Emprunts assumés : image immuable et configuration déclarative (NixOS, ostree), instantanés (ZFS, btrfs).

```
# systeme.toml : la configuration unique
[systeme]
autonomie   = "A1"               # A0..A4, voir la Charte
disques_hote = "lecture-seule"   # ecriture seulement avec consentement

[forge]
modele            = "local"      # local | distant | aucun
classes_autorisees = ["R0", "R1"]
essais_max        = 5
```

10. Démarrage depuis une clé USB

Toutes les architectures visées démarrent en UEFI, et le firmware cherche un exécutable portant le nom standard de sa propre architecture. La clé en porte donc un par architecture.

```
CLE USB (GPT)
+-- ESP  (FAT32, ~512 Mo)
|     +-- EFI/BOOT/BOOTX64.EFI        chargeur x86-64
|     +-- EFI/BOOT/BOOTAA64.EFI       chargeur ARM64
|     +-- EFI/BOOT/BOOTRISCV64.EFI    chargeur RISC-V
|     +-- boot/<arch>/molecule.img    Molécule + services de base (signés)
+-- SYS  (lecture seule)   images système par architecture, en A/B
+-- DATA (persistance)     profils matériels, pilotes forgés, journal, config
```

- **Chargeur** : Limine gère x86-64, aarch64 et riscv64 en UEFI, avec des partitions FAT ; le Secure Boot est possible si l'exécutable est signé et la clé enrôlée. Alternative : un module UEFI maison avec la crate `uefi-rs`.
- **Secure Boot** : trois options : le désactiver, enrôler notre clé, ou passer par une chaîne signée (shim). Les deux premières demandent une action de l'utilisateur.
- **BIOS ancien** : hors périmètre initial (Limine peut le gérer sur x86 si besoin).
- **Sans UEFI** : Apple Silicon, iPhone et beaucoup de téléphones : mode hôte.

Phase	Contenu	Objectif (à mesurer)
1	Firmware UEFI	Hors de notre contrôle
2	Chargeur, puis Molécule copiée en RAM	moins de 1 s
3	Services vitaux (Courants, mémoire, Bassin)	moins de 1 s
4	Profil matériel et pilotes standards, en parallèle	moins de 2 s
5	Expérience adaptative visible	moins de 1 s
Total	Après le firmware	moins de 5 s

Ce sont des objectifs de conception, pas des résultats. Au second démarrage sur la même machine, le profil matériel en cache doit réduire les phases 3 et 4.

11. Performance et fluidité

- Ordonnanceur de type EEVDF avec classes (interactif, temps réel doux, arrière-plan) et budgets par tâche.
- Un exécuteur asynchrone par cœur ; files d'entrée-sortie en anneaux (type io_uring) ; zéro copie entre services.
- Compositeur à balayage direct : l'application affiche sans copie intermédiaire lorsque c'est possible.
- Démarrage parallèle, chargement paresseux des pilotes, profil de machine en cache.
- Mémoire : allocateurs par cœur, pages de 2 Mio ; énergie : états de veille via ACPI.

Métrique	Objectif de conception	Contexte
Interface visible après le firmware	moins de 5 s	Voir section 10
Appels système	environ 20	Zircon : environ 100 prévus ; Asterinas : plus de 210
Aller-retour d'un Courant sur un cœur	quelques microsecondes	Ordre de grandeur des micro-noyaux modernes
Latence entrée vers affichage	moins de 20 ms	Redox a ramené la latence d'entrée USB d'environ 100 ms à 30 ms début 2026
Taille de la Molécule	moins de 15 000 lignes	Objectif de maintenabilité

Règle : des benchmarks reproductibles dès le premier jalon, sur QEMU puis sur matériel réel, et publiés. Sans mesure, « rapide » reste un slogan.

12. Sécurité et modèle de menaces

La base de confiance se limite à la Molécule, la Charte, les portes de la Forge et le module `fondation`. Tout le reste est supposé faillible et confiné.

Menace	Exemple	Parade
Pilote généré défectueux	Plantage, corruption de données	Isolation, IOMMU, watchdog, retour arrière
Injection par les données	Chaîne de firmware ou document contenant des instructions	Entrées traitées comme données ; le modèle n'a aucune clé ; sa sortie passe les portes
Modèle ou dépôt compromis	Code malveillant généré	Portes indépendantes du modèle, signatures, journal
Clé USB modifiée	Image altérée	Images signées ; mesure au démarrage si un TPM existe
Attaque DMA	Périphérique malveillant	IOMMU, fenêtres DMA par clé
Dérive d'auto-évolution	Changements cumulatifs	Portes immuables, budget de changements, revue humaine périodique
Perte de données hôte	Écriture sur un disque interne	Lecture seule par défaut, consentement explicite

13. Applications, runtime et mode hôte

- **Trois formes d'applications** : services natifs Rust (compilés par architecture) ; applications WebAssembly (un seul binaire pour tous les appareils) ; applications hôtes (le même runtime empaqueté pour un autre OS).
- **Mode hôte** : runtime plus pont d'affichage, d'entrée et de réseau vers l'OS hôte. Windows, Linux, macOS et Android permettent une distribution libre ; iOS, iPadOS et HarmonyOS imposent leurs boutiques et des restrictions sur le code dynamique (à valider avant tout engagement).
- **Expérience adaptative** : l'interface est décrite de façon déclarative ; un profil d'appareil (téléphone, tablette, PC, robot avec voix et gestes) choisit l'agencement.

- **Compatibilité** : aucune couche POSIX, Linux ou Windows dans le noyau. Un service POSIX pourra venir plus tard. C'est le choix inverse d'Asterinas, qui garde l'ABI Linux pour faciliter l'adoption.

14. Différenciation par rapport aux OS existants

« Assurer » qu'il soit différent revient à trois choses : cartographier l'existant, choisir où être original, et définir des tests de nouveauté vérifiables. Une garantie absolue est impossible, puisque personne n'a une vue exhaustive ; on organise donc une veille (fin de section). La recherche du 28 septembre 2026 a porté sur Redox, Fuchsia, HarmonyOS, Asterinas, Theseus, Limine et les travaux sur les pilotes générés par modèle de langage.

Système	Noyau	Langage	Appareils visés	Pilotes et auto-évolution	Architectures
Linux	Monolithique modulaire	C, Rust (pilotes)	Serveurs à embarqué	Pilotes écrits par des humains ; recherche LLM externe (AutoDriver, DevGen)	x86-64, ARM64, RISC-V…
Windows NT	Hybride	C, C++	PC, serveurs	Pilotes des constructeurs	x86-64, ARM64
macOS (XNU)	Hybride Mach et BSD	C, C++, Swift	Matériel Apple	Pilotes Apple	Apple uniquement
Android	Linux et runtime	C, C++, Java, Kotlin	Téléphones, tablettes	Pilotes des constructeurs	Selon fabricant
Fuchsia	Micro-noyau Zircon, capacités	C++, Rust	Objets connectés, tablettes, PC	Pilotes en espace utilisateur ; pas d'auto-écriture	ARM64, x86-64
Harmony OS	Micro-noyau distribué (selon les sources)	n/d	Téléphones, tablettes, PC, voitures, IoT	IA applicative ; pilotes Huawei et partenaires	Huawei et partenaires
Redox	Micro-noyau, Unix-like	Rust	Bureau, serveurs	Pilotes en espace utilisateur, écrits par des humains	IA-32, AMD64, AArch64, RISC-V
Asterinas	Framekernel, ABI Linux	Rust	Usage général	Compatibilité Linux ; humains	n/d
Theseus	Cellules dynamiques, évolution à chaud	Rust	Embarqué haut de gamme, edge	Évolution à chaud, pilotée par des humains	n/d
Ce projet	**Micro-noyau immuable (~20 appels), capacités**	**Rust**	**PC en natif, autres en hôte, robots**	**Forge interne sous portes, encadrée par la Charte**	**UEFI x86-64, ARM64, RISC-V, une clé**

n/d : non vérifié lors de la recherche. Windows, macOS et Android : informations générales.

Ce qui est emprunté (et assumé)

Micro-noyau, capacités et absence d'utilisateur ambiant (Fuchsia) ; Rust et pilotes en espace utilisateur (Redox) ; base de confiance minimale (Asterinas) ; composants dynamiques et évolution à chaud (Theseus) ; image immuable A/B (Android, ChromeOS, NixOS) ; protocoles typés (FIDL) ; WebAssembly. Copier ces idées n'est pas un défaut : l'originalité doit se concentrer où elle compte.

Ce qui est nouveau (à démontrer)

Idée	Description	Antériorité trouvée	Test de nouveauté
N1 Forge native	Cycle complet détecter, générer, vérifier, simuler, mode ombre, promouvoir, surveiller, exécuté sur la machine par l'OS, avec classes de risque	Outils externes et hors ligne (AutoDriver, DevGen) pour des pilotes Linux ; AIOS orchestre des agents sans écrire de pilotes ; Theseus fait de l'évolution à chaud manuelle	Existe-t-il un OS qui synthétise, valide et charge lui-même un pilote pour un matériel inconnu ? Aucun trouvé.
N2 Charte	Niveaux d'autonomie appliqués par le noyau aux modifications de l'OS lui-même, portes hors d'atteinte de l'agent	Gouvernance d'agents surtout au niveau applicatif ; l'analyse arXiv:2606.23075 décrit le risque sans OS qui l'applique	Un noyau applique-t-il une politique signée à sa propre auto-modification ? Aucun trouvé.
N3 Clé amphibie apprenante	Image multi-architecture dont le profil matériel et les pilotes forgés s'accumulent sur la clé	Live USB multi-architecture existant, mais sans pilotes forgés ni apprentissage matériel	Une clé s'améliore-t-elle d'une machine à l'autre ? Aucune trouvée.

Honnêteté. N1, N2 et N3 sont des *combinaisons* : aucune brique n'est inédite prise isolément. Ce qui diffère, c'est l'intégration dans l'OS et le niveau de gouvernance. Les Courants (typage, écoute obligatoire, écluses comme primitive) sont une variante de ce que fait déjà Fuchsia, pas une nouveauté. Un grand acteur peut aussi annoncer quelque chose de proche demain ; l'avantage durable viendra d'un système ouvert, minimal et vérifiable.

Veille trimestrielle

- Refaire les trois tests de nouveauté (mots-clés : OS driver synthesis LLM, autonomous kernel self-modification policy, multi-architecture live USB adaptive drivers).
- Suivre Fuchsia, Redox (capacités, ordonnanceur), Asterinas et les travaux type AutoDriver et DevGen.
- Consigner les résultats dans le dépôt du projet, avec date et sources.

15. Feuille de route

Jalons par critères de sortie, sans dates : le rythme dépend du temps disponible.

Jalon	Contenu	Critère de sortie
J0 Fondations	Chaîne Rust multi-cibles, démarrage Limine en UEFI sous QEMU (x86-64 et ARM64), console série	Même image qui affiche un message sur deux architectures
J1 Molécule	Mémoire, tâches, Courants, clés ; deux services qui dialoguent	Tests unitaires et premiers benchmarks
J2 Contenants standards	GOP, virtio, NVMe, xHCI et HID, ACPI de base, IOMMU si disponible	Démarrage sur un vrai PC x86-64 : écran, clavier, stockage en lecture seule
J3 Bassin et clé	Magasin immuable, A/B, persistance, profil matériel	Trois PC différents démarrent avec la même clé
J4 Forge v0 (banc d'essai)	Pipeline complet sur périphériques virtio, hors production	Pilotes R1 générés et validés par les portes sur banc virtuel
J5 Forge intégrée et Charte A0 à A2	Mode ombre sur matériel réel, journal signé, retour arrière	Un pilote R1 forgé, chargé sur vrai matériel, puis promu
J6 Autres architectures	ARM64 réel, RISC-V ; mode hôte sur Linux, Windows, Android	Mêmes services en natif et en hôte
J7 Robots et Apple, Huawei	Pont ROS 2 pour humanoïdes ; hôte sur macOS, iOS, HarmonyOS selon les règles	Démo contrôlée, R3 verrouillé

Conseil de tri : ne pas réinventer ce qui n'est pas la différence. Évaluer la réutilisation d'OSTD (Asterinas, licence MPL), des crates de rust-osdev et de virtio-drivers, et garder l'effort pour la Forge, la Charte et la clé amphibie.

16. Risques et décisions ouvertes

Risque	Gravité	Parade
Ampleur des pilotes	Élevée	Standards d'abord ; réutiliser des crates ; Forge en accélérateur
Fiabilité de la génération	Élevée	Portes indépendantes du modèle ; essais bornés ; simulation avant matériel
Sécurité de la Forge	Élevée	Section 12 ; Charte ; portes immuables
Secure Boot et signatures	Moyenne	Documenter l'enrôlement de clé ; prévoir un mode démonstration
Matériel fermé	Moyenne	Mode hôte ; ne pas promettre le natif partout
Charge pour une personne seule	Élevée	Jalons courts ; réutilisation ; QEMU d'abord ; publier tôt

Décisions à prendre

- Le nom du projet et sa licence (MPL, Apache-2.0, MIT ; le choix conditionne la réutilisation d'OSTD).
- Le modèle de la Forge : local, distant ou les deux ; niveau de connexion accepté.
- Le premier matériel réel de test (un PC x86-64 récent est le plus simple).
- Le niveau d'autonomie par défaut (A1 proposé) et qui détient la clé de signature.
- Tout écrire ou réutiliser OSTD et des briques existantes.

Annexe A. Glossaire

Terme	Sens
Molécule	Micro-noyau immuable et signé
Courant	Canal typé, avec quota, que l'on peut écouter
Écluse	Quota et contre-pression d'un Courant
Clé	Capacité : droit explicite, restreignable et révocable
Contenant	Pilote isolé en espace utilisateur
Profil matériel	Graphe des périphériques détectés, mis en cache sur la clé
Bassin	Stockage immuable, A/B, instantanés, persistance
Vapeur	Cache en RAM, recalculable
Forge	Développeur interne : génère, teste, promeut, surveille les pilotes
Charte	Politique signée d'autonomie, appliquée par le noyau
Mode ombre	Essai sur matériel réel avec périmètre limité
Natif, hôte	Le système est l'OS, ou une application sur un autre OS

Annexe B. Références

- Redox OS : Wikipedia (plateformes, micro-noyau) ; « This Month in Redox », janvier 2026, redox-os.org/news/this-month-260131/ (capacités, EEVDF, latence USB).
- Fuchsia : Wikipedia, « Fuchsia (operating system) » (Zircon, ARM64 et x86-64, version F31 de juillet 2026) ; documentation Zircon sur fuchsia.googlesource.com.
- HarmonyOS : huaweicentral.com, « HarmonyOS 6 reached 66 million devices » (2026) ; techjunction.co/encyclopedia/harmonyos.
- Asterinas : Peng et al., « Asterinas: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB », USENIX ATC 2025 ; github.com/asterinas/asterinas.
- Theseus : github.com/theseus-os/Theseus ; Kevin Boos, « Theseus, a new Rust OS » (présentation).

- Limine : github.com/limine-bootloader/limine (architectures, Secure Boot).
- AutoDriver et DriveBench : Kharlamova et al., « LLM-Driven Kernel Evolution: Automating Driver Updates in Linux », arXiv:2511.18924.
- DevGen : Wang et al., « Bridging Kernel Drivers and Virtual Device Models with LLM-Powered Automation », ACL 2026, System Demonstrations (2026.acl-demo.24).
- Sécurité des systèmes auto-évolutifs : « Safety in Self-Evolving LLM Agent Systems: Threats, Amplification, and Case Studies », arXiv:2606.23075.
- AIOS : « AIOS: AI-Integrated Operating System » (résumé consulté sur emergentmind.com).